

ThatNamelessGame Detailed Design Document

Last updated: 2026-10-05. This document covers the current implementation in `lib/` and `test/`. Details of CPU decision-making are separated into `cpu.md`.

1. Architecture



Layer	Implementation	Responsibility
Startup	<code>main.dart</code> , <code>initialization/</code>	Flutter initialization, dependency preparation, retry screen
Rules	<code>game/game_engine.dart</code>	Immutable game positions, legal moves, position transitions, end determination, initial positions
Match	<code>play/play_session.dart</code>	State machine, timer, countdown, CPU presentation, result persistence
CPU	<code>cpu_strategy.dart</code> , <code>cpu_scheduler.dart</code>	Move decisions and presentation planning; see <code>cpu.md</code> for details
Settings	<code>settings/</code>	Settings values, progress, serialized persistence
Audio	<code>audio/</code>	BGM/SE preloading, playback state, lifecycle integration
UI	<code>screens/</code>	Screen transitions and rendering <code>PlayState</code>

Game rules do not depend on Flutter/UI. Treat both `GameSession` and settings values as immutable objects, and never change past positions during animation or search.

2. Startup

`startApplication()` configures binding, landscape lock, immersive display, and exception handlers. `Bootstrap._initialize()` does the following.

- Obtains settings through `SettingsRepository.create()` and `loadOrCreate()`.
- Runs `AudioController.create()` and `preloadScreenImages()` in parallel.
- Uses `AudioObserver.attach()` to apply initial settings and foreground state to audio.
- Exposes a `ProviderContainer` overriding the `SettingsStore`, initial settings, and `AudioController`.

On failure, dispose created containers/audio and show the retry screen. `dispose()` releases the observer, container, and audio.

3. Game Rules

On each turn, select one non-zero hand belonging to the current player and one non-zero opponent hand. The target value becomes $(\text{target} + \text{attacker}) \% 5$; hands with value 0 cannot be selected.

Type/API	Description
<code>GameRules</code>	The standard is three hands and modulus 5.
<code>GamePosition</code>	Near/far hands, turn, and rules. Values are 0..4. <code>key</code> is a position key encoding hands and turn.
<code>GameMove</code>	Attacker and target <code>HandPosition</code> .
<code>GameSession</code>	Position, set of already-seen keys, and an optional result.
<code>legalMoves</code>	Returns all combinations of non-zero current-player hands and non-zero opponent hands.
<code>isLegalMove</code>	Verifies that the game is unfinished, indices are in range, and both attacker and target are non-zero.
<code>applyMove</code>	Throws <code>ArgumentError</code> for an illegal move. Returns a new position and does not modify the original match.

`applyMove` determines the end state in this order: (1) if the target side has no surviving hands, the attacking side wins by elimination; (2) if the updated key has already appeared, end by loop (the side with more surviving hands wins; equal counts are a draw); (3) otherwise add the key to history and continue.

`InitialPositionGenerator` uses `InitialHands.common`, and `SoloInitialPositionGenerator` uses `InitialHands.solo`. Both randomize hand display order. The former also randomizes sides and the first player.

4. Match State Machine

`PlaySessionNotifier` is an `autoDispose` provider that exposes `PlayState` to the UI. `PlayState` holds mode, session, phase, remaining time, countdown, selected hands, attacking/paused state, result, and operation generation `operationId`.

```

countdown --4 seconds--> selecting --confirm--> attacking --800 ms--> selecting
                                     |                               |
                                     |--Home--> confirmingExit  |--game end-->
finished

```

Operation	Conditions and processing
<code>startCountdown(id)</code>	Starts when the generation matches, the phase is countdown, and the app is foregrounded; plays SE.
<code>select(side, hand)</code>	Sets a living hand as attacker or target only while selecting during a human turn.
<code>confirm()</code>	Moves to attacking when both selections are made and legal.
<code>requestExit/cancelExit</code>	Shows/dismisses exit confirmation. Stops/resumes the timer only for applicable human turns.
<code>setForeground</code>	Applies elapsed progress before backgrounding, then resets the clock. Background time is not added.
<code>commitResult</code>	Persists a finished result to settings exactly once.
<code>replay(id)</code>	Resets to a new position when finished and the generation matches.

At 600 ms into an attack, switch to an intermediate position that updates only the target's appearance. At 800 ms, switch to the complete position after `GameEngine.applyMove()`. If the game is over, play the result sound; otherwise begin the next turn.

Do not constantly poll. `_scheduleTick()` schedules only the next meaningful boundary with `Timer`: the next second boundary for countdown; 600/800 ms while attacking; `CpuScheduler.untilNext` for CPU; and a timer-display update or deadline for a human turn. Do not advance while not foregrounded, confirming exit, finished, or during the pre-start countdown.

5. Modes

Mode	Initial position	Human side	Limit	Result sound
twoPlayer	common	near/far	Setting value (Unlimited allowed)	<code>win</code> regardless of winner; <code>draw</code> for a draw
easy	common	near	30 seconds	<code>win</code> for player win, <code>lose</code> for CPU win, <code>draw</code> for a draw
normal	solo	near	15 seconds	Same as above
hard	solo	near	7 seconds	Same as above

Pause the game timer on CPU turns, then confirm after the selection presentation. `cpu.md` is authoritative for CPU decisions, evaluation, and tie-break rules.

6. Settings and Progress

`AppSettings` holds language, BGM/SE state, active time limit, stars for each mode, and progress counters. `AppSettingsNotifier` immediately updates state, while `_SettingsSaveQueue` serializes `SharedPreferences` saves in arrival order. BGM/SE changes are also applied to `AudioController`.

For two-player, increase stars and the count after every completed match. For solo, increase stars and win streak on victory; preserve them on a draw; reset the streak to 0 on defeat or interruption. A star is yellow for a streak of five or more, blue when earned, and clear otherwise.

`SettingsRepository` validates persisted values for type and range, falling back to defaults for invalid values. It saves the initialized flag only after all values are saved successfully.

7. Audio and Assets

`Assets` centrally manages paths for screen images, BGM, and SE. `SoundEffect` represents only the type, and `Assets.soundEffect(effect)` returns the relative path for `AssetSource`.

`AudioController.create()` preloads one BGM player and one player per SE. If audio-context setup fails, it is silent; if an individual load fails, only that sound is disabled.

Feature	Rule
BGM	Loop only when all conditions are met: enabled, foregrounded, no result sound, and no stop owner.
Normal SE	Serializes stop → resume through the operation queue.
Countdown SE	Uses a queue separate from normal SE. Invalidates old scheduled events by generation number.
Result SE	Stops BGM while playing, then synchronizes against the latest conditions after completion, stop, or disposal.
Lifecycle	<code>AudioObserver</code> notifies foreground state.

8. UI and Quality

Home renders mode selection, stars, and screen transitions. Settings updates settings. Rules has nine pages (GIFs replay from the beginning each time). Play renders the match, overlays, attacks, and result.

`TappableImage` has an in-progress guard and cooldown.

Tests separate rules, initial positions, CPU, state machine, audio ordering, settings persistence, startup, and screen operations. `FakeClock`, replaceable providers, and `FakeAudioPlayer` avoid reliance on real time or the real audio SDK. Non-fatal settings, audio, and CPU failures are recorded by `AppErrorHandler`; CPU search failure falls back to the first legal move.